

PDF

Scenario Description - L...

Worked for 41s >

I have reviewed the PDF and created a detailed  **implementation_plan.md** artifact that outlines a comprehensive, phase-by-phase approach for building the LinkedIn Clone with Next.js, strictly typed TypeScript, and MongoDB.

Regarding your request for the development environment, once we set up the `docker-compose.yml` file, the command you will use to start MongoDB in dev is:

bash

```
docker-compose up -d
```

(This will run MongoDB and potentially Mongo-Express in the background.)

Please review the implementation plan in the Artifact window. I have included a few architectural questions (like your preference between Vanilla CSS vs TailwindCSS, and Image Storage strategy) in the **Open Questions** section.

Let me know your thoughts or hit **Proceed** on the plan if everything looks good, and we'll start executing Phase 1!

Implementation Plan

Implementation plan for Phase 6: Messaging & Endorsements.

←  0 Files With Changes

 Review Changes

✓ Proceeded with  Implementation Plan • 7 

Worked for 1m >

I'm currently initializing the Next.js project with Tailwind CSS in a temporary folder (`linkedin-clone`), and it is installing the necessary dependencies. This will take just a moment.

Once it's done, I will:

1. Move the generated files into the main directory (`Fcode`).
2. Install Material UI (`@mui/material` , `@emotion/react` , `@emotion/styled`).
3. Set up the `docker-compose.yml` for MongoDB.
4. Configure `.env` and `.gitignore` .
5. Set up the Mongoose connection utility and our basic layout components (Navbar, Sidebar).

I will resume automatically once the initialization is complete!

Initialize Next.js project finished >

Worked for 2m >

I have completed the Phase 1 setup as requested!

Here is what was accomplished:

1. Created the Next.js project using Tailwind CSS and TypeScript in the root directory.

←  0 Files With Changes

 Review Changes

I have completed the Phase 1 setup as requested!

Here is what was accomplished:

1. Created the Next.js project using Tailwind CSS and TypeScript in the root directory.
2. Installed Material UI (`@mui/material`, `@emotion/react`, `@emotion/styled`) and its required icons.
3. Created the `docker-compose.yml` to spin up MongoDB and Mongo Express locally.
4. Created the `.env` with initial values and set up the robust Mongoose connection utility inside `src/lib/db.ts`.
5. Created the initial UI shell for the LinkedIn clone, utilizing a sticky `Navbar` at the top and a standard grid layout consisting of a `Sidebar` and main feed block inside `src/app/page.tsx` and `layout.tsx`.

You can view the full details in the [📖 Walkthrough Artifact](#).

To see it in action, you can run the following two commands in your terminal:

```
bash
```

```
docker-compose up -d  
npm run dev
```

Would you like to move on to Phase 2 (User Authentication & Profile Setup) next?

🔧 Task

Task list for Phase 6: Messaging & Endorsements.

← 📄 0 Files With Changes

📄 Review Changes

Walkthrough

Final walkthrough describing the completion of the Messaging and Skills systems.



✓ Proceeded with  **Implementation Plan** · 1 

Worked for 3m >

I have completed Phase 2!

Here is what was accomplished:

1. **User Model:** Created the strictly-typed Mongoose `User` model, including sub-schemas for Skills, Education, and Experience.
2. **NextAuth Integration:** Configured `NextAuth.js` with a `CredentialsProvider` and set up the session handling logic.
3. **Authentication Pages:** Built the `/register` and `/login` pages using Material UI components for a polished, LinkedIn-like experience. Passwords are securely hashed with `bcryptjs`.
4. **Profile Page:** Built a dynamic profile page at `/profile/[id]`. If you log in and visit your own profile, you will see inline edit buttons allowing you to update your Name, Headline, Location, and About section.
5. **Navbar Update:** Wrapped the app in a NextAuth `SessionProvider` and updated the `Navbar` to display your user icon and a Sign Out button when logged in, or a Sign In

←  0 Files With Changes

 [Review Changes](#)

the session handling logic.

3. **Authentication Pages:** Built the `/register` and `/login` pages using Material UI components for a polished, LinkedIn-like experience. Passwords are securely hashed with `bcryptjs`.
4. **Profile Page:** Built a dynamic profile page at `/profile/[id]`. If you log in and visit your own profile, you will see inline edit buttons allowing you to update your Name, Headline, Location, and About section.
5. **Navbar Update:** Wrapped the app in a NextAuth `SessionProvider` and updated the `Navbar` to display your user icon and a Sign Out button when logged in, or a Sign In button when logged out.

You can view the full details in the updated  **Walkthrough**.

Since your development server is already running, you can test this out right now by navigating to: <http://localhost:3000/register> 

Would you like me to proceed to **Phase 3: Connections & Follow System**?

Task

Task list for Phase 6: Messaging & Endorsements.

Walkthrough

Final walkthrough describing the completion of the Messaging and Skills systems.

7:39 PM



🔧 Task

Task list for Phase 6: Messaging & Endorsements.

📖 Walkthrough

Final walkthrough describing the completion of the Messaging and Skills systems.



Recoverable Error

Hydration failed because the server rendered HTML didn't match the client. As a result this tree will be regenerated on the client. This can happen if a SSR-ed Client Component used:

- A server/client branch `if (typeof window !== 'undefined')`.
- Variable input such as

Worked for 1m >

I've addressed the hydration mismatch error!

Here's what caused it and how I fixed it:

1. **MUI with Next.js App Router:** Material UI uses Emotion for styling, which can inject styles dynamically causing hydration mismatches in Next.js Server Components. I fixed this by installing `@mui/material-nextjs` and wrapping your root layout with `<AppRouterCacheProvider>`. This ensures styles are injected properly during Server-Side Rendering (SSR).
2. **NextAuth Session State:** The NextAuth `useSession` hook returns `status: "loading"` on

← 📄 0 Files With Changes

📄 Review Changes

Recoverable Error

Hydration failed because the server rendered HTML didn't match the client. As a result this tree will be regenerated on the client. This can happen if a SSR-ed Client Component used:

- A server/client branch `if (typeof window !== 'undefined')`.
- Variable input such as

Worked for 1m >

I've addressed the hydration mismatch error!

Here's what caused it and how I fixed it:

- 1. MUI with Next.js App Router:** Material UI uses Emotion for styling, which can inject styles dynamically causing hydration mismatches in Next.js Server Components. I fixed this by installing `@mui/material-nextjs` and wrapping your root layout with `<AppRouterCacheProvider>`. This ensures styles are injected properly during Server-Side Rendering (SSR).
- 2. NextAuth Session State:** The NextAuth `useSession` hook returns `status: "loading"` on the server and initial client render, but immediately resolves. However, the conditions were mismatching causing React to panic. I added a `mounted` state to `Navbar.tsx` so the server and client output match perfectly on the very first render, resolving the error.

Your dev server should automatically hot-reload and the error overlay should be gone. Can you verify if the app is rendering correctly now? Let me know if you are ready to move on to Phase 3!

7:44 PM



```
Registration error: TypeError: Cannot read properties of undefined (reading 'json')
  at POST (src\app\api\auth\register\route.ts:22:27)
20 |
21 |   if (existingUser) {
```

Worked for 22s >

I've caught the bug and fixed it!

The issue was a slightly incorrect import in the new Next.js App Router API format:

`NextResponse` must be imported from `"next/server"` rather than `"next"`. I've updated both the `register/route.ts` and the `users/[id]/route.ts` API routes to use the correct import.

You can try submitting the registration form again—it should work perfectly now! Let me know if you run into any other issues or if you're ready to proceed to the next phase.



Give a common schema for while sending the response. Fix it now

7:53 PM



Worked for 1m >

Give a common schema for while sending the response. Fix it now

Worked for 1m >

I have created a standard, common response schema and applied it to our existing endpoints. Here is the standardized schema I implemented in `src/lib/apiResponse.ts`:

```
json
{
  "success": true,    // boolean
  "message": "...",  // string
  "data": { ... },   // optional payload
  "error": { ... }   // optional error details
}
```

I've refactored both `/api/auth/register` and `/api/users/[id]` to utilize this standard format via helper functions (`sendSuccess` and `sendError`). I also updated the frontend profile fetching code to properly consume the new `.data` payload structure.

Everything should be working smoothly now with the new schema! Let me know if you run into any other bugs, or if you're ready to proceed to Phase 3 (Connections).

7:53 PM



Give alert or notification system in the UI when some action is success or fail. Fix it now . As example when sign up it should give a message about the process whether it is success or fail

← 0 Files With Changes

Review Changes

Give alert or notification system in the UI when some action is success or fail. Fix it now . As example when sign up it should give a message about the process whether it is success or fail

Worked for 1m >

I have successfully implemented a global alert and notification system!

To do this:

1. I created a robust `NotificationContext` using Material UI's `Snackbar` and `Alert` components.
2. I wrapped our entire application with this `NotificationProvider` inside `Providers.tsx`.
3. I integrated the `showNotification` hook into the **Registration**, **Login**, and **Profile Edit** processes.

Now, when you try to sign up, it will immediately display a green success popup at the top of your screen if the registration works, or a red error popup containing the exact issue (e.g., "User already exists with this email") if it fails. The same applies to logging in or saving profile changes.

Feel free to try creating an account now to see the notification in action! Let me know if you are ready to proceed with the next phase of the implementation.



For the UI throughout the application use a proper colour theme. Use white as the background and blue as the primary colour and light blue as intermediate colour other things. Fix this now

Worked for 1m >

I have updated the application's global color theme to exactly match your requirements! Here's what I changed:

- 1. Material UI Theme Setup:** I configured a centralized Material UI `ThemeProvider` inside our `Providers.tsx`.
 - **Primary Color:** Set to a professional, strong blue (`#0a66c2`).
 - **Intermediate Color:** Set to a light blue (`#e8f3ff`).
 - **Background Default:** Set to pure white (`ffffff`).
- 2. Global Layout Background:** I explicitly changed the Tailwind class in `layout.tsx` from the off-white `bg-[#f3f2ef]` to `bg-white` so the entire app rests on a pure white background.
- 3. Button Colors:** I swept through the Registration, Login, and Profile pages and removed the hardcoded Tailwind blue backgrounds (`bg-blue-600`). The buttons now properly inherit their styles dynamically from the new centralized Material UI blue theme.

The UI should look much cleaner and brand-aligned now! Let me know if everything looks correct on your end.

5 files changed +30 -7 >

 Review

<  0 Files With Changes

 Review Changes



Worked for 1m >

I completely understand now. In the previous implementation, although the Material UI buttons were updated to the primary blue color, the layout still retained many of the default gray backgrounds, gray borders, and multi-colored text elements (like the red/green/orange "Start a post" buttons you shared in the screenshot).

I have swept through the entire application to strictly enforce your "White/Blue/Light Blue" theme everywhere:

1. **Navbar:** Removed the red "Sign Out" button, turning it into a primary blue outline button. The search bar now has a light blue background, and the navigation icons are a unified blue instead of gray.
2. **Sidebar & Profile:** The cover photos now default to a solid primary blue instead of gray. All gray borders have been replaced with a soft light blue (`border-blue-100`). The text colors and hover states are all mapped to variations of blue and light blue (`bg-blue-50`).
3. **Feed (Home Page):** I stripped away the green, orange, and red colors from the "Start a post" actions. Everything now uniformly utilizes the blue primary and light blue hover theme. The "LinkedIn News" section also uses the new color palette for borders, backgrounds, and text.

Check the dev server now. The entire interface should feel deeply saturated in the white and blue corporate theme you specified. Let me know what you think!

Next you have to create a common end point to uploads and separate end point for all downloads. It should be support for when uploading image for profile and upload image for posts as well. So the particular images should be in the relevant places according to the local file structure. Use suitable design pattern as well. consider security aspects as well.

Worked for 24s >

I have created a comprehensive Implementation Plan for a secure, local file upload and download system using the Strategy Design Pattern.

Please review the proposed plan in the  **Implementation Plan** artifact, particularly the **Open Questions** regarding file size limits and public vs private access for the download endpoint. Let me know what you think and if I can proceed!

Implementation Plan

Implementation plan for Phase 6: Messaging & Endorsements.

✓ Proceeded with  **Implementation Plan** • 5 

Worked for 2m >

I have successfully implemented the secure Upload and Download system according to our approved plan!

What's been built:

- **Strategy Pattern** (`src/lib/upload/`): We now have a robust `UploadStrategy` interface with `ProfileUploadStrategy` (2MB limit) and `PostUploadStrategy` (5MB limit).
- **Upload Endpoint** (`POST /api/upload`): Fully protected by NextAuth, it safely parses the `FormData`, validates the image, and uses the correct strategy to save the file locally into `uploads/profiles/` or `uploads/posts/`.
- **Download Endpoint** (`GET /api/download/[...path]`): Strictly locked down to logged-in users only. It features advanced path-traversal protection so attackers cannot escape the `uploads/` directory to read sensitive files (like `.env`).
- **Profile Picture Upload UI**: The `ProfilePage` is fully wired up!

Please verify this is working:

1. Go to your app in the browser.
2. Navigate to your Profile page (`http://localhost:3000/profile/[your-id]`).
3. Hover your mouse over your Avatar. A "**Change Photo**" overlay will appear.

←  0 Files With Changes

 **Review Changes**

Please verify this is working.

1. Go to your app in the browser.
2. Navigate to your Profile page (`http://localhost:3000/profile/[your-id]`).
3. Hover your mouse over your Avatar. A "**Change Photo**" overlay will appear.
4. Click it, select an image from your computer, and watch it securely upload and render!

I have detailed the entire architectural setup in the updated 📖 **Walkthrough**.

Let me know if the upload flow works perfectly on your end, or if you'd like to move on to the next phase (Connections & Following)!

🔖 Task

Task list for Phase 6: Messaging & Endorsements.

📖 Walkthrough

Final walkthrough describing the completion of the Messaging and Skills systems.



Now give me the next implementation plan phase wise the tasks which have to complete.

Worked for 23s >

I have drafted the next implementation plan!

As requested, I included a brief overview of the remaining roadmap, and then broke down

Phase 3: Connections & Networking into specific, actionable steps.

< 📄 0 Files With Changes

📄 Review Changes

Now give me the next implementation plan phase wise the tasks which have to complete.

Worked for 23s >

I have drafted the next implementation plan!

As requested, I included a brief overview of the remaining roadmap, and then broke down **Phase 3: Connections & Networking** into specific, actionable steps.

Please review the  **Implementation Plan** artifact. Specifically, take a look at the **Open Questions** section regarding whether we should build a separate "Follow" system or keep it strictly to Connections for now.

Let me know your thoughts and if you approve this plan to begin execution!

 Implementation Plan

Implementation plan for Phase 6: Messaging & Endorsements.



✓ Proceeded with  **Implementation Plan** · 2 

Worked for 2m >

Phase 3 is now complete!

The entire Connections and Follow system has been built from the ground up, with full API integration, database models, and dynamic UI updates across the **Profile**, **Network**, and **Sidebar** components.

You can review a detailed explanation of what was built and how to test it in the updated

Summary

←  0 Files With Changes

 Review Changes

✓ Proceeded with  Implementation Plan • 2 

Integration, database models, and dynamic updates across the `Profile`, `Network`, and

`Sidebar` components.

You can review a detailed explanation of what was built and how to test it in the updated

 Walkthrough.

Key Highlights:

- **Dynamic Profile Buttons:** Visiting another user's profile will automatically detect if you are connected, pending, or not connected, and adjust the action buttons (`Connect` -> `Pending` -> `Accept` -> `Message`) accordingly.
- **Dedicated Network Hub:** A `/network` page lists all pending connection invitations, active connections, and suggests random unregistered users.
- **Auto-Following:** A dedicated `Follow` model runs under the hood. Connecting with someone automatically adds a mutual follow relationship.
- **Live Sidebar Stats:** Your sidebar component will dynamically calculate and fetch your exact connection count!

Go ahead and test sending and receiving a connection request by creating a second account in an incognito window as suggested in the Walkthrough! Let me know if it all works correctly, and if you are ready to begin Phase 4 (Posts & Feed).

Task

Task list for Phase 6: Messaging & Endorsements.

Walkthrough

Final walkthrough describing the completion of the Messaging and Skills systems.

10 files changed +686 -40 >

 Review

Worked for 1m >

Phase 4: Posts, Feed & Interaction is fully implemented and live!

I have built out the entire content-sharing system. You can review the specifics of what was done and how to test it in the updated  **Walkthrough**.

Key Features Implemented:

- **Post Creation:** You can now create text posts and attach images directly from the Home Feed. Media attachments securely utilize our backend upload strategies.
- **Dynamic Feed:** The feed aggregates posts from you **and** any user you are following (which includes your connections), sorted chronologically.
- **Dedicated Commenting System:** We opted for the dedicated **Comment** model as requested. You can expand a post to add and read comments in real-time.
- **Interactive Likes:** Clicking "Like" instantly increments the counter and toggles the icon state.

Go ahead and create your first post from the Home page! Try attaching an image and dropping a comment on it.

Once you are satisfied that the feed, posts, and interactions are working perfectly, let me know, and we can move on to **Phase 5: Search & Notifications!**

Task

Task list for Phase 6: Messaging & Endorsements.

Walkthrough

Final walkthrough describing the completion of the Messaging and Skills systems.

←  0 Files With Changes

 **Review Changes**

✓ Proceeded with  **Implementation Plan** · 2 

Worked for 4m >

Phase 5 is complete!

The application is now deeply interactive with real-time WebSockets, Pagination, and Search functionality. I have updated the  **Walkthrough** with the technical specifics of how everything connects.

Important Note for Testing: Because I configured a separate `socket.io` server to run alongside Next.js for WebSockets, I updated your `package.json`'s `dev` script to use `concurrently`. **Please stop your currently running `npm run dev` terminal and run it again** so that both Next.js and the socket server start up properly!

Once restarted, you can test everything:

1. Try searching for another user in the Navbar search box.
2. If you open two browser windows side-by-side (logged into different accounts), liking a post or sending a connection request from one will instantly trigger a popup alert and increment the red notification bell on the other!
3. Scroll down the Home feed to see the Infinite Scroll load more posts automatically.

Let me know if the real-time events are functioning correctly for you, and we can move on to the final stretch: **Phase 6: Messaging & Endorsements!**

🔧 **Task**

Task list for Phase 6: Messaging & Endorsements.

←  0 Files With Changes

 **Review Changes**

Build Error

Worked for 16s >

I apologize for that syntax error! A missing closing brace `}` in

`src/components/Providers.tsx` caused the build to fail.

I've just added it back, and the Next.js dev server should automatically recompile successfully now. Please verify if the app is back up and running!



give me the next implementation plan

Worked for 29s >

I have generated the  **Implementation Plan** for the final phase—**Phase 6: Messaging & Endorsements!**

This phase will focus on real-time private messaging using the WebSockets we set up in the previous phase, as well as a profile skills and endorsement system.

Please review the plan, specifically the **Open Questions** regarding:

1. Message history pagination (load all vs last 50).
2. Implementing Read Receipts (seen status).

Let me know your thoughts and if you approve!

Make the readme file including all the implemented features in here. It should contain include a README.md (or similar) with:

- o Assumptions I have made about the scenario/requirements
- o Instructions to run the project (setup steps, dependencies, commands,

Worked for 31s >

I have successfully created the `README.md` file at the root of your project!

The document is formatted beautifully and comprehensively covers:

- **Implemented Features:** A breakdown of the auth, social graph, smart feed, real-time websockets, and infrastructure.
- **Key Architectural Decisions:** Explaining *why* we separated connections from follows, why we used `socket.io` concurrently, and how the Upload Strategy pattern works.
- **Assumptions Made:** Detailing our thoughts regarding local storage, unauthenticated access controls, and session strategies.
- **Instructions to Run:** A step-by-step guide detailing the required prerequisites, dependencies, environment variables, Docker setup, and the concurrent dev command.

You can review the full contents of the file directly! Let me know if there's anything you would like to add or modify.

using websocket implemented real time message notification and real time chat

Worked for 21s >

I have updated the `README.md` to explicitly highlight that **using WebSocket implemented real time message notification and real time chat!**

You can review the updated file here: [M+ README.md](#)

Let me know if there's anything else you'd like to adjust or add!

1 file changed +1 -0 ▾

 Review

[M+ README.md](#) /c:/Users/sasmi/OneDrive/Documents/Fcode



using websocket implemented real time message notification and real time chat. This feature is not working up to now implement it

Worked for 54s >

Ah! I apologize for that.

When I added the real-time chat functionality, I updated the `socket.js` WebSocket server with new event handlers for messages (`send_message` , `receive_message` , etc). However, the standalone Node background task for the WebSocket server was still actively running the *old* code from Phase 5 in memory, so it didn't know how to route your chat messages!

I have just forcibly **killed the old socket server** and **restarted it** so that it now uses the newly